UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Operating Systems

Lab experience - Unix Sockets

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Lab experience on Unix Sockets

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Goals of this lab experience:

- Getting used to UNIX sockets

Warmup

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Review the source files `guess-race-socket-server.c` and `guess-race-socket-client.c` that you can find in `Moodle` → `Esercitazioni e dimostrazioni` → `Laboratorio socket`
- They are yet another implementation of the ”guess race” game, but this time using UNIX sockets for inter-process communication instead of pipes.
- Read it, understand how it works, compile and run it, check that it works as expected

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Guess and guess again...

Exercise 1: no SIGUSR1 anymore!

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Modify `guess-race-socket-server.c` and `guess-race-socket-client.c`. The server should not use SIGUSR1 to signal the start of a round.
- Hint: make the server send a new type of message to the clients to tell them that a new round started!
- Client should wait for this message before sending the next move, and the server should wait for both clients to send their move before starting a new round. You can reuse the existing code for process creation and inter-process communication through sockets, but you will need to modify the game logic to implement the new communication scheme.

Exercise 2: let the clients know the score!

- Now modify them again to let the clients know:
  - who won the round and what are the current scores
  - who won the game
- Hint: make the server send a new type of message to the clients to tell them the result of the round and the game!
- Client should terminate gracefully when the game ends.

Exercise 3: why just 2 players?

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Now modify them again to let the server handle a variable number of clients. The number of clients required to start a game should be the first command-line parameter of the server
- When a client connects, the server should tell him how many clients are connected and how many more clients are needed to start the game.
- When the required number of clients is reached, the server should start the game and send a message to all clients to tell them that the game has started.

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Concurrent server

Warmup

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Review the source files `concurrent-server.c` and `concurrent-client.c` that you can find in
  `Moodle` → `Esercitazioni e dimostrazioni` → `Laboratorio socket`
- Read it, understand how it works, compile and run it, check that it works as expected

Exercise 4: concurrent primality test

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Modify `concurrent-server.c` and `concurrent-client.c` to implement a concurrent primality test server. The server should listen for incoming connections from clients, and for each client it should create a new process to handle the client's requests. Each client should send a number to the server, and the server should reply with whether the number is prime or not.